Timeouts and retries

2026-08-16. Numbers are the shipped `config/tuning.yaml`*.*

Four independent layers can each abandon work, and each has its own retry. They nest, and their retries **multiply** — which is the part that is easy to get wrong when tuning any single number.

Two layers now treat a slow call as *suspect* rather than as work in progress: the CALL layer (safechain only, 40s) and the SCREEN phase (both backends, 10s). The second exists because the first cannot reach it — see "Why screen needs its own fence".

(Diagrams are plain ASCII on purpose: box-drawing glyphs are outside Courier's WinAnsi encoding and render substituted or blank in the PDF.)

The layers

```
+= TURN ========================================  turn_wall_clock_s = 360s =+
| asyncio.wait_for on the whole turn. On expiry the coroutine is CANCELLED,  |
| so every agent still in flight dies with it -- they report `cancelled`,    |
| not a timeout of their own.                       no retry: the turn ends  |
|                                                                            |
| +- PHASE ------------------------ one fence per kind of agent ----------+ |
| |  screen        30s  (stall fence 10s, INSIDE the 30 -- see below)      | |
| |  orch_plan / reviewer   25s        specialist          240s            | |
| |  distiller    120s      distiller_drain  60s   report_agent  150s      | |
| |                                                                        | |
| |  Expiry = that agent failed. The turn continues, degraded.             | |
| |  Two retry here: orch_plan (the round-1 watchdog), and screen, which   | |
| |  abandons a wedged attempt at 10s and re-issues within its own 30s.    | |
| |                                                                        | |
| | +- AGENT --------------------- _MAX_SPECIALIST_ATTEMPTS = 2 --------+  | |
| | | Re-runs the whole agentic loop when the ANSWER is wrong:          |  | |
| | |   ungrounded_retry   built on a failed tool call                  |  | |
| | |   absence_reread     denies what the tools returned               |  | |
| | |   max_turns_retry    blew its turn budget                         |  | |
| | |                                                                   |  | |
| | | +- CALL ------------ one HTTP request to the model ----------+    |  | |
| | | |  stall fence  40s  ->  abandon, re-issue ONCE              |    |  | |
| | | |  full budget  60s  ->  TimeoutError + safechain_retry_     |    |  | |
| | | |                        stalled (the falsifying datum)      |    |  | |
| | | |  HTTP 401          ->  refresh token, retry IN PLACE       |    |  | |
| | | |  403 / 400         ->  FirewallRejection, escalates to the |    |  | |
| | | |                        guidance loop below                 |    |  | |
| | | +------------------------------------------------------------+    |  | |
| | +-------------------------------------------------------------------+  | |
| +-----------------------------------------------------------------------+ |
+===========================================================================+

  +- REJECTION (wraps the call) ------ firewall.max_retries = 2 --------+
  |  403 compliance block / 400 bad request are DETERMINISTIC, so an     |
  |  identical retry is guaranteed to fail the same way. This loop       |
  |  retries with a CHANGED input instead -- `_inject_guidance` appends  |
  |  firewall guidance telling the model what to remove. 401 is the      |
  |  opposite case (a stale token, transient) and is retried in place.   |
  +----------------------------------------------------------------------+

  Also in the path, but not a fence: the firewall semaphores
  (specialist 8, orchestrator 4).
```

The call layer, in detail

Added 2026-08-13, for safechain only. The screen phase carries the same idea at a different layer (see below); everything about the reasoning is shared.

```
issue request
     |
     +-- returns < 40s --------------------------> ANSWER  (p99 is 7.5s)
     |
     +-- still running at 40s
            |  the call is not slow, it is WEDGED
            |  log: safechain_call_stalled
            v
         abandon it   (ainvoke is genuinely cancellable, so the
                       request aborts -- no orphaned thread)
            |
         issue a FRESH request, full 60s budget
            |
            +-- returns ------------------------> ANSWER    (~45s total)
            |
            +-- stalls too ---------------------> FAILS at 100s
                   |                          logs safechain_retry_stalled
                   +-- exactly one extra attempt; never a loop
```

The budget bets that the retry ESCAPES. The alternative was a budget above the 126-131s plateau so a stalled retry could ride it out — safer, but it makes every persistent wedge cost ~170s. The bet is supported by concurrent divergence: `distiller.bureau` returned in 5.8s while `distiller.modeling` hung 120s in the same pool at the same moment, so the wedge belongs to the REQUEST and a fresh one should not inherit it. Given that, the budget only has to cover a healthy call — p99 7.5s in dev, 2.1-5.6s in prod, so 60s leaves ~8x margin.

	before	after
retry escapes the stall	~130s	**~45s**
retry stalls too	~130s	fails at 100s, logged
call is healthy	unchanged	unchanged, issued once

The bet is falsifiable, and recorded in two places because they have different readers. The session JSONL carries `safechain_call_stalled` and `safechain_retry_stalled` for a human reading one run. The node trace carries the same thing as tags, because AgenticEval scores from the trace DB and never reads the JSONL — a stall visible only in the log would be invisible to every scored run:

```
node        specialist.modeling.round_1
tags        ["stall_retry"]           call abandoned and re-issued
            ["stall_retry_failed"]    the re-issue ALSO timed out
extra_json  {"stall_retry_after_s": 40, "retry_budget_s": 60}
```

Tagged on the ROUND, so a stall attributes to a specific agent. Zero `stall_retry_failed` means the budget can drop further; anything non-trivial means the wedge survives re-issue and the budget must go back above the plateau.

Why 40s and not tighter. Per-call latency is p99 7.5s over 88 dev calls and 2.1-5.6s in prod, so 40 is ~5x the observed worst case: it cannot abandon a healthy call, and it only governs how long a STALL burns before the retry.

`SAFECHAIN_STALL_RETRY_S=0` disables it. The first attempt is clamped to `min(stall_fence, full_budget)`, so lowering `SAFECHAIN_CALL_TIMEOUT_S` really does lower it — unclamped, the fence silently outlived a tighter setting.

Why this layer exists

Measured in the private env: safechain calls do not run slow, they **stall**. In one turn `distiller.bureau` returned in 5.8s while `distiller.modeling` — concurrent, same pool, same model — hung. Every stall allowed to finish landed in a narrow band:

```
orchestrator.round_2  125.98s ok       spend_payments  129.50s ok
spend_payments        129.83s ok       crossbu         130.67s ok
```

while every failure died at **its own phase fence**, not at a natural duration:

```
report_agent          100.01s   <- report_agent_s was 100, now 150
distiller.modeling    120.01s   <- distiller_s = 120
general_specialist     25.00s   <- orch_plan_s = 25, round_1 ran 0.00s
```

So the fence a phase happens to carry decided whether it survived, while normal calls in those same turns took 2-13s. Widening fences only converts a failure into a 130s wait; escaping the stall is what recovers the time.

Why screen needs its own fence

Added 2026-08-16, after "stuck at question check" recurred. The call-layer retry above **cannot reach this phase**: its stall fence is 40s and the whole screen budget is 30s, so the phase fence always fires first and the reviewer gets "question check took too long" having had exactly one attempt.

```
the ordering, before
     0s        10        20        30              40
     |---------|---------|---------|---------------|
     screen phase budget ......... X               |
                                   ^               |
                      phase dies here              |
                                   call-layer stall fence would
                                   have fired HERE -- unreachable

after: the fence moves INSIDE the phase
     0s        10                                  30
     |---------|-----------------------------------|
     attempt 1 X  abandon + re-issue with what is left
               ^
          10s stall fence
```

Three properties, each chosen to avoid re-tuning anything else:

- **The two attempts SHARE the 30s** — the retry gets `30 - elapsed`, so the worst case is unchanged and the existing `screen_timeout` handler still fires on the same `TimeoutError`.
- **It sits at the PHASE, not the call**, so it works on both backends. The call-layer retry is safechain-only; a stall in dev would never have been covered by it.
- **10s is ~2x the phase's own <5s target**, so a healthy screen never trips it. `SCREEN_STALL_RETRY_S=0` disables it. The first attempt is clamped to `min(fence, budget)` — the same bug that had to be fixed in the call layer.

Same tag vocabulary as the call layer, deliberately, so AgenticEval greps one set of names: `stall_retry` / `stall_retry_failed` on the `screen` node, plus `screen_stalled` / `screen_retry_stalled` in the JSONL.

It **retries a stall, it does not fix one.** The re-issue sends the same prompt, so a wedge caused by prompt size will reproduce; `stall_retry_failed` is what distinguishes that from a transient backend hiccup, and `PRIOR_QUESTIONS_FOR_SCREEN` (12) is the knob for the former.

What a retry costs

An abandoned attempt is not free, and where the cost lands depends on where the wedge is — which is the same unresolved question the call-layer bet rests on.

wedge is...	attempt 1 tokens	what the retry adds
at the provider (inference started)	**billed** — cancelling closes your side, it does not un-run the work	a second full prompt
before the provider (pool, TLS, socket)	nothing sent, nothing billed	a second full prompt

So worst case is ~2x input and up to 2x output; best case is 1x. For screen the exposure is small (question + <=12 prior questions + the relevance skill), and for a specialist round it is whatever that round's context is.

The trace under-reports this. `attach_usage` records the prompt excerpt *before* the request but the token COUNTS only *after* the response returns (`firewall_client.py:101` vs `:140`). A cancelled call never reaches the second call, so an abandoned attempt contributes **zero tokens to node_trace** while potentially costing real money. AgenticEval scores from the trace, so its token totals are a floor, not a total, on any run where stalls occurred.

Nothing is rebuilt for the retry except the coroutine and the HTTP request. The agent, the shim, `_CLIENTS.firewalled_client`, the httpx pool and (on safechain) the cached LangChain model are all reused — deliberately, since rebuilding a client would add DNS, TLS and token acquisition to a path already in trouble. The consequence: **the retry is fresh at the request level, not below it.** If the wedge lives in the connection or the pool rather than the request, a retry can inherit it.

The multiplication trap

Agent-level and call-level retries compose. A specialist that retries once, each of whose calls stalls once, issues **up to 4 requests** — and its phase fence has to hold all of them. That is why the call-level budget is exactly one extra attempt rather than a general retry loop: the arithmetic has to stay inspectable against the fence above it.

Known inconsistencies

- **RESOLVED — the "phase fence must be tighter than the per-call cap" invariant was backwards for multi-round agents.** It holds for single-call phases (`screen`, `orch_plan`), where a tighter fence gives the clearer error. A specialist making four calls must be LOOSER than one call's cap or it can never complete more than one slow call. The rule is now split by phase kind, and `specialist_s`/`report_agent_s` are sized to hold one worst-case call (40 + 140 = 180) plus the agent's own rounds.
- **RESOLVED — a phase tighter than the call fence gets no retry at all.** The flip side of the above, and it cost real failures before anyone noticed: `screen` at 30s sits under the 40s call-layer stall fence, so that retry was unreachable code for this phase. Fixed by giving the phase its own 10s fence rather than by loosening the phase. `orch_plan` (25s) has the same shape and is covered by its round-1 watchdog; **any new single-call phase under 40s needs one or the other**, and nothing tests for that.
- **The reviewer shares `ORCH_PLAN_TIMEOUT_S` with the round-1 watchdog**, and they want opposite things: planning wants a tight 25s so a stall aborts and retries fast; the reviewer wants to outlast a stall. One knob cannot serve both — the reviewer is the agent that failed at exactly 25.00s.
- **The documented "slowest realistic path" does not fit the turn fence.** `30 + 25 + 240 + 25 + 150 = 470s` against `turn_wall_clock_s = 360`. The guard test checks each phase against the fence individually, so nothing catches the sum.